EF Core: Filtering Data with Where()

Concise summary • Light editor-style code blocks

1. Core Idea

Where() in LINQ corresponds to **WHERE** in SQL. EF Core translates the condition so the database filters rows before returning results.

Filtering.cs — C#

```csharp
context.Students.Where(s => s.Status == "Active");
```

Query.sql — SQL

```sql
SELECT * FROM Students WHERE Status = 'Active';
```

2. Filter Before Execution

Build the query first. **ToList()** executes it; **ToQueryString()** previews SQL without executing the query.

GetActiveStudents.cs — C#

```csharp
var query = context.Students
    .Where(s => s.Status == "Active")
    .OrderBy(s => s.StudentId);

// Show generated SQL
PreviewSQLUsingToQueryString(query.ToQueryString());

// Execute query
var students = query.ToList();
```

3. Avoid Filtering in Memory

Wrong — loads all data first

WrongExample.cs — C#

```csharp
var students = context.Students.ToList();
var activeStudents = students
    .Where(s => s.Status == "Active")
    .ToList();
```

Correct — filter in the database

CorrectExample.cs — C#

```csharp
var students = context.Students
    .Where(s => s.Status == "Active")
    .ToList();
```

4. Key Takeaways

- Filter before calling **ToList()** when possible.
- Database filtering reduces unnecessary data transfer and memory use.
- Use **ToQueryString()** to inspect generated SQL.
- Next topic: **Projection with Select()** — retrieve only needed columns.